CS 470: Assignment 03

Programming Assignments (80%)

Your goal is to detect the bounding boxes for **white blood cells** in the BCCD dataset.

WARNING: You will have to run the script Prepare_A03.py to download the data.

A03.py

In this module, write a class called **CellFinder** with the following functions:

- **def __init__(self, model_dir)**
  - Constructor for the class
  - Any saved models can be loaded from a file in model_dir if desired.
  - I recommend saving model_dir as an instance variable: **self.model_dir = model_dir**
- **def train_WBC(self, train_data)**
  - If training is desired, use train_data to train any useful information/models for finding white blood cells and save it to self.model_dir.
  - Look at General_A03.predict_dataset for how to loop through and use train_data.
  - That said, train_data should only contain white blood cell bounding boxes for the ground truth.
  - **WARNING: If your saved model is larger than 15MB, do NOT commit it to your repo!**
    - **Instead provide instructions in your repo to download the model elsewhere.**
  - If training is not needed or desired, keep the function but use this command: **pass**
- **def find_WBC(self, image)**
  - Given a single color image:
    - Find the white blood cells (which tend to have **blue blobs in the center** and to be **fairly large**)
    - Compute the bounding box (ymin, xmin, ymax, xmax) for each white blood cell found
    - Return a list of detected bounding boxes

The way you approach the problem(s) above is largely up to you. It is assumed that this will be done via some manner of color segmentation.

Please note, if you use machine learning / deep learning, you are ONLY allowed to train on the training dataset, NOT the testing dataset!

Unlike some of the other assignments, **you are permitted to use OpenCV, Numpy, Scikit-Learn, scipy, and even PyTorch functionality**, including:

- numpy
    - where
    - expand_dims
    - sum
    - sqrt
    - argmin
    - flatten
    - reshape
- skimage.segmentation
    - slic
- cv2
    - mean
    - kmeans
    - connectedComponents
    - floodFill
    - cvtColor
    - Any of the edge detection approaches
    - Any of the Hough approaches
    - Any of the morphological approaches

Prepare_A03.py

This script must be run ONCE to download all of the BCCD data (graciously available from: https://github.com/Shenggan/BCCD_Dataset)

General_A03.py

This is the core functionality for the training and evaluation scripts.

Train_A03_WBC.py

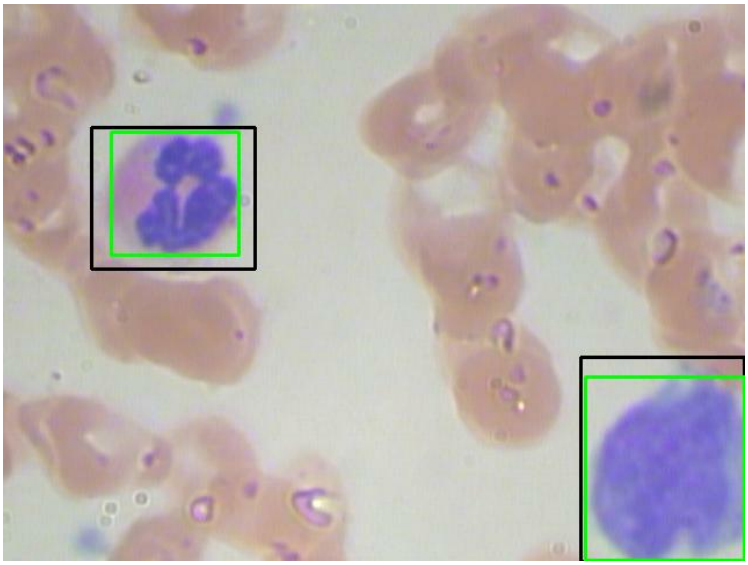
This script runs training for your class on the white blood cell training data.

Eval_A03_WBC.py

This script runs evaluation on how well your approach works finding white blood cells on the training and testing data.

- Loads the BCCD dataset into training and testing sets (for white blood cells).
- Creates an "assign03/output_wbc" directory.
    - If it already exists, the program will ask you if you want to overwrite it.
- Calls your prediction approach on the training and testing images, computing the accuracy of the count as well as the IOU (Intersection-Over-Union) of the predicted bounding boxes.
- It also saves the output images with ground truth (black) and predicted (green) boxes.

- Prints the metrics as well as saves them to a file, assign03/output_wbc/RESULTS_WBC.txt.



Results (10%)

After running the training programs, copy your result txt file(s) to the MAIN directory of your repository.

README (10%)

Update the README.md of your repository with the following:

- Add a new entry under "Runnable Python Scripts" for "A03.py". Briefly describe your approach(es) for training and finding the cells.
- If your saved model(s) are larger than 15MB, include instructions on how to download the models. You may use external storage sites to provide the models (e.g., OneDrive).

BONUS POINTS

For the entire assignment, bonus points will be awarded to the submission that has the following:

- (+%5) Best **white blood cell testing** IOU in class (provided it is over 0.4)

Example Approach for White Blood Cells

The example results were generated using the following approach (you are free to use this approach, or part of it, or none of it):

- **(Step 1) Get superpixel groups (to honor color and edge boundaries).**
    - Let's assume the image with superpixel indices is given by *segments*.
    - Make sure to set *start_label* parameter to zero!
    - The number of groups found can then be extracted using:
        - cnt = len(np.unique(segments))
- **(Step 2) Compute mean color per superpixel**
    - Create a numpy array to hold the mean color per superpixel:
        - group_means = np.zeros((cnt, 3), dtype="float32")
    - Loop through each superpixel index (i.e., for specific_group in range(cnt)):
        - Create mask images using np.where:
            - mask_image = np.where(segments == specific_group, 255, 0).astype("uint8")
        - Add the channel dimension back in to the mask using np.expand_dims()
        - Use cv2.mean (with mask) to compute mean value per group
            - Note that cv2.mean() returns a 4-channel value, so you will want to slice the result:
                - group_means[specific_group] = cv2.mean(image, mask=mask_image)[0:3]
- **(Step 3) Use K-means on GROUP mean colors to group them into 4 color groups.**
    - Let's assume cv2.kmeans() returned the following:
        - *ret* = not used
        - *bestLabels* = list of which superpixel mean colors map to which kmeans group
            - E.g., if superpixel group 3's mean color fits into kmeans group 1, then:
                - bestLabels[3] → 1
        - centers = kmeans group center values
            - E.g., if kmeans group 1 has a color of (2,56,120), then:
                - centers[1] → (2, 56, 120)
- **(Step 4) Find the k-means group with mean closest to:**
    - **White blood cells → blue (255,0,0) (REMEMBER: BGR for OpenCV)**
    - E.g., find the color in *centers* closest to target color
    - Can use Euclidean distance color segmentation
- **(Step 5) Set that k-means group to white and the rest to black.**
    - I.e., we only want to keep the group closest to target color
    - E.g., if group 1 was the closest to target color, then this step should give you:
        - centers[0] → (0,0,0)

        - centers[1] → (255,255,255)
        - centers[2] → (0,0,0)
        - centers[3] → (0,0,0)
- **(Step 6) Determine the new colors for each superpixel group**
    - Convert *centers* to unsigned 8-bit
    - Get the new superpixel group colors:
        - colors_per_clump = centers[bestLabels.flatten()]
- **(Step 7) Recolor the superpixels with their new group colors (which are now just white and black)**
    - You can use numpy broadcasting and indicing to create a new image *cell_mask*:
        - cell_mask = colors_per_clump[segments]
    - **Convert cell_mask to grayscale!**
- **(Step 8) Use cv2.connectedComponents to get disjoint blobs from cell_mask.**
    - Slide 14 of region slide deck
    - Remember this function returns:
        - retval = number of connection components / blobs found
        - labels = image with labels marked
- **(Step 9) For each blob group (except 0, which is the background):**
    - Get the COORDINATES of the pixels that belong to that group using np.where:
        - coords = np.where(labels == i)
    - As long as coordinates are found, get the min and max x and y coordinates to get the bounding box (ymin, xmin, ymax, xmax).
    - Add bounding box to list of bounding boxes.

You absolutely do NOT have to use this exact approach. You may opt for a subset of this idea (e.g., just use k-means or superpixels), or you may use a completely different approach.

I will recommend though that you take a good look at np.where, which can be very useful: https://numpy.org/doc/stable/reference/generated/numpy.where.html

Grading

Your OVERALL assignment grade is weighted as follows:

- 80% - Programming assignments
- 10% - Results
- 10% - README
- (+5%) - BONUS

I reserve the right to take points off for not meeting the specifications in this assignment description.

The following specific penalties (applied to the WHOLE assignment) can be expected:

Problem	**Maximum Penalty**
Code that is not syntactically correct	60
Hardcoded such that only one white blood cell can be predicted	10
Way too many cells predicted for white blood cells	10
Poorly fitting bounding boxes for white blood cells	10
Used training approach, but either model(s) not committed OR missing instructions anywhere on how to acquire model(s)	10
Used training approach, but white blood cell training is coded incorrectly	10
Missing results file(s) in main directory	10
README update missing/incorrect	10
Not hardcoded, but never finding more than one white blood cell	5
Used training approach, but model does not load correctly for testing (if the model doesn't load at all, a much harsher penalty will be enacted).	5
Training function missing (even with pass)	5
Technically correct code, but poor code design	5
Missing assignment Git branch	5

In general, these are things that will be penalized:

- **Code that is not syntactically correct (up to 60 points off!)**
- Sloppy or poor coding style
- Bad coding design principles
- Code that crashes, does not run, or takes a VERY long time to complete
- Using code from ANY source other than the course materials
- Collaboration on code of ANY kind; this is an INDIVIDUAL PROJECT
- Sharing code with other people in this class or using code from this or any other related class
- Output that is incorrect
- Algorithms/implementations that are incorrect
- Submitting improper files
- Failing to submit ALL required files